

C语言程序设计

第七讲 函数与模块化 函数



张 华

函数

程序的模块化

函数

- 定义

- 原型

- 调用

- 参数传递

函数的嵌套调用

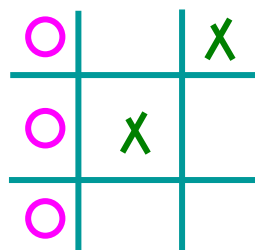
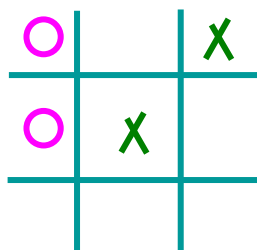
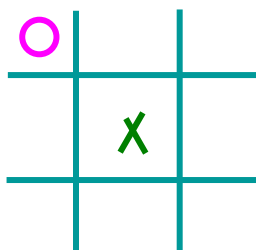
程序设计举例

引例

问题：“井”字棋游戏程序

游戏规则：

- 玩家和计算机轮流下棋子。
- 每次一方只能在某一个空格处下一颗棋子。
- 胜负判断：
 - 若棋盘的某一行，或某一列，抑或某一对角线上的三个格子被某一方的棋子占据，则该方胜利；
 - 否则，为平局。



程序模块化

分而治之与程序的模块化

- ✿ 把一个规模较大的问题分解成若干子问题，对每一个子问题编写一段程序（即程序**模块**（**module**））。
- ✿ 然后，通过组合模块来构建完整的程序。
- ✿ 这些模块比只包含单一模块的复杂程序更容易实现和管理。
- ✿ 这些模块可以被重复使用。

“井”字棋程序

主控

初始化棋盘

显示棋盘

玩家走一步

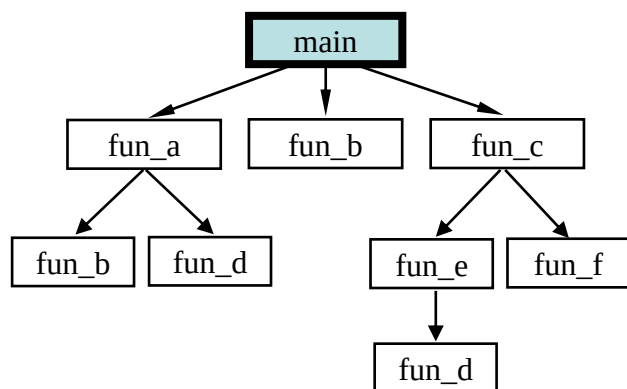
计算机走一步

判断胜负

C语言的函数

在C语言中，函数（function）是构成程序的基本模块。

- 一个C程序由一个或多个函数组成，有且仅有一个主函数，即main()函数。
- 每个函数完成一个相对独立的且功能明确的任务。
- 由主函数调用其他函数，其他函数也可以互相调用。
- 同一个函数可以被一个或多个函数调用任意多次。



从模块化的角度看一个C语言程序的结构

为什么使用函数实现程序的模块化

提高程序开发的效率

- 促成小组或团队开发。
- 程序易于管理。

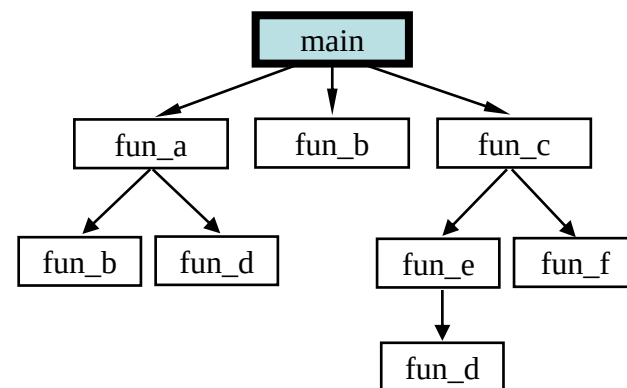
代码重用

- 使用函数作为构件来创建程序。
- 函数可以被重复使用。

抽象

- 扩大代码的应用范围。
- 隐藏了函数的实现细节。

➤ 例如，我们没看过经常使用的库函数`printf()`的实现代码。



C语言函数的类别

C语言的函数有两大类：

✿ 标准库函数

- C系统提供了丰富的函数，按类型以库为单位组织。
- 例如
 - 数学计算函数：`sqrt()`，`abs()`
 - 输入/输出函数：`scanf()`，`printf()`

✿ 自定义函数

- 程序员可以编写函数来完成特定的功能。
- 例如，主函数`main()`

对于开发人员

- ✿ 尽量避免从零开始构建一切。
- ✿ 尽可能多的熟悉C系统中的标准函数库。

案例分析：一个简单的函数

编写和使用一个简单的函数 (cw0801.c)

- 定义一个函数square，用来计算任意整数的平方。
- 然后，使用该函数计算从1到10所有整数的平方。

```
#include <stdio.h>
```

```
int square(int y) {  
    return y*y;  
}
```

定义函数

```
void main() {  
    int x;  
    for (x=1; x<=10; x++)  
        printf("%d ", square(x));  
}
```

使用函数

1 4 9 16 25 36 49 64 81 100

函数的定义

定义函数的格式

函数类型 函数名 (参数表)

{

函数体语句

}

函数头

函数体

✿ 函数名：一个有效的标识符。

✿ 参数表：声明参数，参数间用逗号分隔。

参数1类型 参数1名称, 参数2类型 参数2名称.....

- 接收传递进来的数据。
- 必须为每个参数单独指定数据类型。
- 但 **int** 可以省略。

✿ 函数类型：返回值的类型说明符。

- 如果不指定，默认是 **int**。
- **void**：表示函数不返回任何值。

函数的定义

定义函数的格式

函数类型 函数名 (参数表)

{

函数体

}

✿ 函数体：包括声明语句和可执行语句。

➤ 在函数体内可以声明变量。

➤ 不能在函数体内定义函数，即函数的定义不允许嵌套。

✿ 控制返回：结束执行，把程序的控制交还主调函数，也可以用**return**语句返回一个数值。

➤ }

➤ **return**;

➤ **return** 表达式;

无返回值

有返回值

案例分析：函数的设计与定义

函数的设计与定义 (cw0802.c)

✿ 定义函数找出三个数中的大数。

```
#include <stdio.h>
```

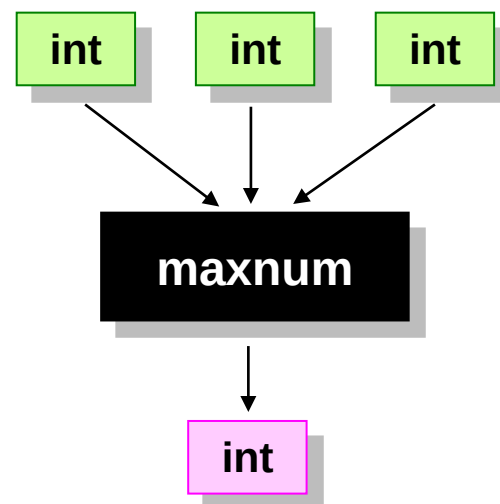
```
int maxnum(int, int, int);
```

函数原型

```
void main() {  
    int a, b, c;  
    printf("Input three integers: ");  
    scanf("%d%d%d", &a, &b, &c);  
    printf("Maximum is: %d\n", maxnum(a, b, c));  
}
```

```
int maxnum(int x, int y, int z) {  
    int max = x;  
    if (y>max) max = y;  
    if (z>max) max = z;  
    return max;  
}
```

接口 (interface)



函数原型

函数原型

函数类型 函数名 (参数表) ;

✿ 用来对函数进行声明。

✿ 编译器使用函数原型来检查函数调用的合法性。

✿ 注意：函数原型要与函数的定义一致。

✿ 例如

```
int maxnum(int a, int b, int c);
```

```
int maxnum(int, int, int);
```

```
int abs(int x);
```

```
double fabs(double x);
```

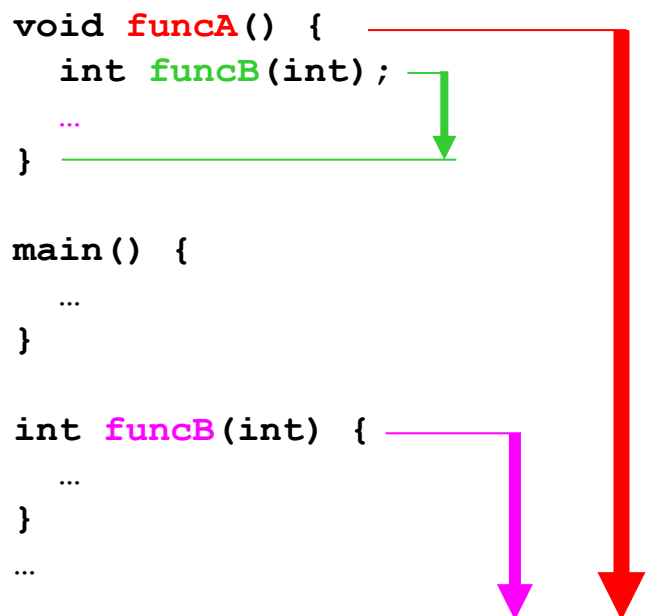
```
double pow(double x, double y);
```

参数名可以省略

函数原型

函数原型在程序文件中的位置不同，函数名的作用范围（可以使用该函数的代码范围）不同。

- ✱ 在所有函数的外面
- ✱ 在函数内部



```
void funcA() {  
    int funcB(int);  
    ...  
}  
  
main() {  
    ...  
}  
  
int funcB(int) {  
    ...  
}  
...
```

The diagram illustrates the scope of function names in the provided code. A red arrow originates from the opening curly brace of `funcA()` and points down to the end of the `main()` function, indicating that `funcA` is visible throughout the entire program. A green arrow originates from the `funcB(int)` declaration inside `funcA()` and points down to the end of the `funcA()` function block, indicating that this `funcB` is only visible within `funcA`. A purple arrow originates from the opening curly brace of `funcB(int)` and points down to the end of its own function block, indicating that this `funcB` is only visible within its own function.

函数原型

如果程序中没有包含函数原型。

- 编译程序会使用该函数第一次出现的情形来形成自己的函数原型。
 - 函数的定义
 - 函数的调用
- 默认情况下，编译程序假定函数返回int型的结果，但不会对参数进行任何假定。
 - 如果传递给函数的参数不正确，编译程序不会检查到这些错误。

```
main() {  
    f(2.5);  
}  
  
f(x) {  
    return x*x;  
}
```

warning C4013: 'f' undefined; assuming extern returning int

得不到正确的结果

函数原型

函数原型强迫参数采用正确的数据类型。

举例

➤ `printf("%.3f", sqrt(4) );`

`double sqrt(double);`

➤ 函数原型使编译程序把整数值4转换为double型的值4.0

✱ 与函数原型中的参数类型不完全对应的参数值会在调用函数之前被转换成合适的数据类型。

➤ 遵循C语言的类型提升规则。

函数原型与头文件

标准库的头文件

- ✿ 每个标准库函数都有对应的头文件。
 - 包含了标准库中所有函数的函数原型，
 - 以及那些函数所需的数据类型和常量的定义。
- ✿ 使用**#include**命令把头文件包含到程序文件中：
 - **#include** <文件名>
 - 例如，**#include** <math.h>

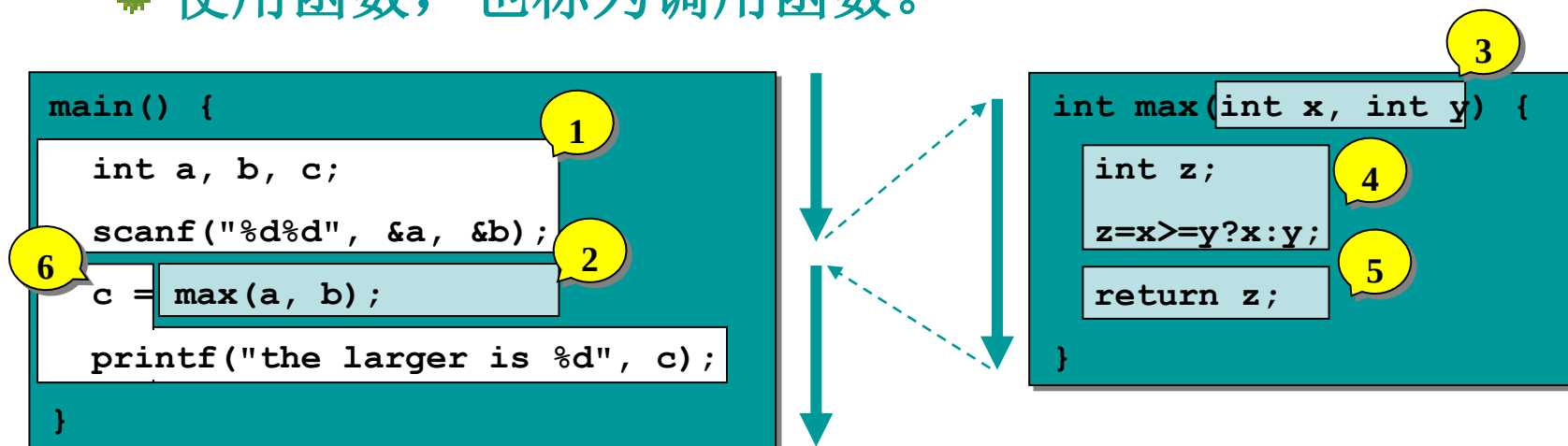
程序员可以创建自己的头文件

- ✿ 使用.h扩展名。
- ✿ 使用下面的命令格式包含头文件：
 - **#include** "文件名"
 - 例如，**#include** "abc.h"

函数调用

函数调用

✿ 使用函数，也称为调用函数。



- 1、执行主调函数中的语句。
- 2、主调函数暂停，保存现场。
- 3、把实参的值拷贝给形参，控制权交给被调函数 `max`。
- 4、执行函数 `max` 中的语句。

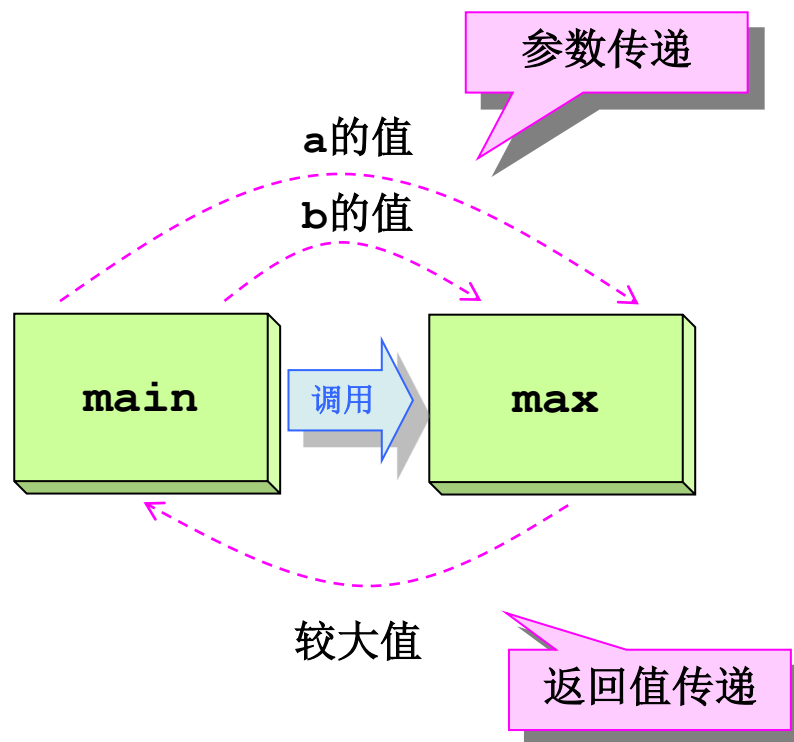
- 5、被调函数执行结束，把函数值返回给主调函数。
- 6、恢复现场，把控制权还给主调函数，主调函数继续执行。

参数传递

函数间的数据传递方式：

- 参数
- 返回值

```
int max(int x, int y) {  
    ..... return z;  
}  
main() {  
    ..... c=max(a, b);  
}
```



实参和形参

实参和形参

```
int max(int x, int y)
{ int z=x>=y?x:y;
  return z;
}

main()
{ int a, b, c;
  scanf("%d%d", &a, &b);
  c=max(a, b);
}
```

形式参数

简称“**形参**”。在**函数定义时**表示可以接受传递过来的值。

实际参数

简称“**实参**”。在**函数调用时**给出。

形参

形参

```
int max(int x, int y)
{ int z=x>=y?x:y;
  return z;
}

main()
{ int a, b, c;
  scanf("%d%d", &a, &b);
  c=max(a, b);
}
```

❖ 形参必须是变量。

❖ 只有在函数被调用、启动后，才临时为其分配存储单元，并接受主调函数传来的数据。

❖ 在函数调用结束后，形参所占存储单元被释放。

实参

实参

```
int max(int x, int y)
{ int z=x>=y?x:y;
  return z;
}

main()
{ int a, b, c;
  scanf("%d%d", &a, &b);
  c=max(a, b);
}
```

❖ 实参是函数调用时主调函数传送给被调函数的形式参数的实际值。

❖ 实参必须有确定的值。

❖ 实参可以是常量、变量和表达式。

参数传递

参数传递的顺序

```
void show(int a, int b)
{
    printf("a=%d, b=%d\n", a, b);
}
main()
{
    int x=6, y;
    show(++x, x);
}
```

a=7, b=6

❖当实参表中有多个实参时，对实参的求值顺序并不确定。

❖VC是按从右往左的顺序求值。

在参数传递时

b=x
a=++x

❖Dev-C++呢？

参数传递

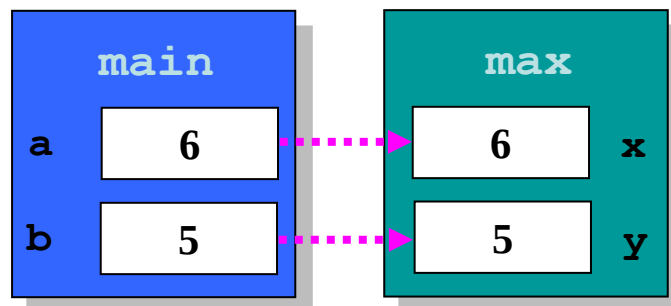
参数传递对参数值的影响

```
int max(int x, int y)
{ int z=x>=y?x:y;
  x++; y++;
  return z;
}

main()
{ int a=6, b=5, c;
  c=max(a, b);
  printf("%d,%d,%d",a,b,c);
}
```

6,5,6

- ❖ 实参与形参不共用存储单元。
- ❖ 参数传递时，把实参的值赋值给形参。
- ❖ 形参值的变化不影响实参的值。



值传递和引用传递

■ 计算机语言的函数间参数传递方式有两种类型。

✿ 值传递

- 主调函数把实参的值传递给被调函数的形参。
- 在被调函数内改变形参的值不会改变主调函数中实参变量的值。
- 如果函数不需要修改实参的值，就采用这种调用方式。

✿ 引用传递

- 主调把实参变量“自身”传递给被调函数的形参。
- 在被调函数内改变形参的值将改变主调函数中实参变量的值。
- 另一个优点是数据传递效率高。

■ 在C语言中，所有参数传递都采用值传递。

参数传递

实参和形参的类型应该相同或赋值兼容。

```
int max(int a, int b)
{ int c=a>=b?a:b;
  return c;
}
main()
{ int x=6, y=5, z;
  z=max(x, y);
  printf("%d", z);
}
```

- 如果实参x, y是整型, 则结果正确;
- 如果实参x, y是字符型, 则自动进行类型转换, 结果正确;
- 如果实参x, y是短整型, 则自动进行类型转换, 结果正确;
- 如果实参x或y是浮点类型, 则自动进行类型转换, 有数据丢失, 结果可能不正确。

b=y

a=x

在参数传递时

函数的返回值

函数返回的值的类型与函数的类型一致。

✱ 如果不一致，采用函数的类型，对返回值进行类型转换。

```
int f(double x)
{
    return x;
}
```

x 的类型?
返回值的类型?

```
main()
{ double y;
  y = f(2.5);
  printf("%f", y);
}
```

f(x) 的类型?
f(2.5) 的类型?

数组做函数的参数

把数组作为参数传递给函数

```
int myArray[20];  
myFunction(myArray, 20);
```

- 实参用数组名
- 通常要把数组的长度传递给函数

对应的函数定义

```
void myFunction(int array[], int arraySize) {  
}
```

长度在这儿不起作用

通过一个参数接受长度

- 实参与形参的类型相同
- 可以不指定形参数组长度

函数原型中数组参数的形式

```
void myFunction(int[], int);
```

可以有空格

数组做函数的参数

把数组作为参数传递给函数

举例

```
void try(int array[ ], int len)
{ int i;
  for(i=0; i<len; i++) array[i] *= 2;
}

void main()
{ int i, list[5] = {0,1,2,3,4};
  try(list, 5);
  for(i=0; i<5; i++) printf("%d ", list[i]);
}
```

0 2 4 6 8

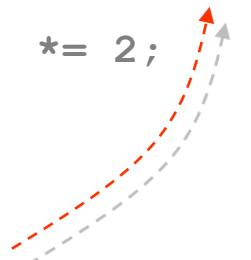
数组做函数的参数

把数组作为参数传递给函数

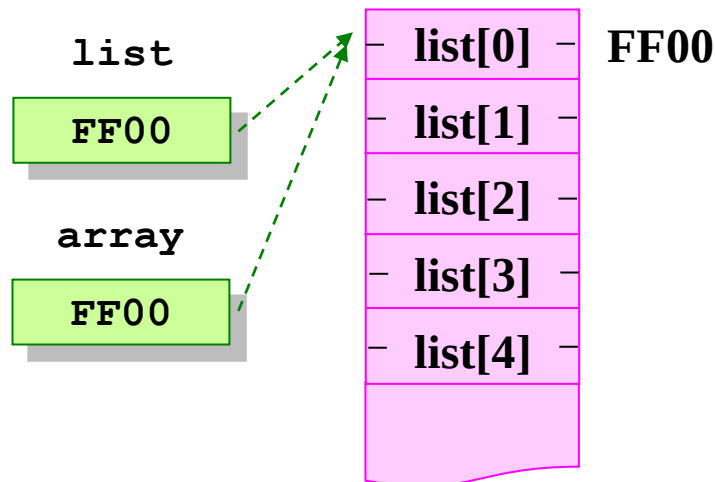
数组的传递模拟了引用传递

```
void try(int array[ ], int len)
{ ... array[i] *= 2;
}

void main()
{ ... try(list, 5);
}
```



数组名是数组第一个元素的地址
(内存单元编号)。所以，依然是按值传递 (传递的是地址)



数组做函数的参数

数组名

✿ 数组名是一个指针，其值是数组第一个元素的地址。

```
#include <stdio.h>

void main() {
    int array[10];

    printf("array = %p\n"
           "&array[0] = %p\n",
           array, &array[0]);
}
```

```
array = 19b8
&array[0] = 19b8
```

%p

输出地址的格式说明字符。

printf的格式控制字符串可以分多行写，但每一行必须用“”括起来。

数组元素做函数调用的实参

数组元素可以做函数调用的实参

- ✿ 数组元素就是一个普通的变量。

- ✿ 数组元素不能做函数的形参。

➤ 因为不能单独定义数组元素，即函数参数不可能是数组元素。

举例

- ✿ 源代码 (cw1006.c)

```
#include <stdio.h>

void modifyArray(int [], int);
void modifyElement(int);

void main() {
    int a[] = {0, 1, 2, 3, 4}, i;
```

数组元素做函数调用的实参

举例

源代码

```
printf("\nThe values of the original array are:\n");  
for (i=0;i<5;i++) printf(" %d", a[i]);  
modifyArray(a, 5);  
printf("\nThe values of the modified array are:\n");  
for (i=0;i<5;i++) printf(" %d", a[i]);  
  
printf("\nThe value of a[3] is %d", a[3]);  
modifyElement(a[3]);  
printf("\nThe value of a[3] is %d", a[3]);  
}
```


数组元素做函数调用的实参

举例

源代码

```
void modifyArray(int array[], int size) {  
    int j;  
  
    for (j=0;j<size;j++)  
        array[j] *= 2;  
}  
  
void modifyElement(int e) {  
    printf("\nValue in modifyElement is %d", e *= 2);  
}
```

数组元素做函数调用的实参

举例

运行结果

```
The values of the original array are:
```

```
0 1 2 3 4
```

```
The values of the modified array are:
```

```
0 2 4 6 8
```

```
The value of a[3] is 6
```

```
Value in modifyElement is 12
```

```
The value of a[3] is 6
```

二维数组做函数的参数

二维数组做函数的参数

数组元素做函数的实参

➤ 值传递

二维数组名做函数的实参，对应的形参是二维数组

➤ 模拟引用传递

```
void exchange(int a[][3], int b[][2], int m, int n);  
void main() {  
    int a[2][3], b[3][2];  
    ...  
    exchange(a, b, 2, 3);  
    ...  
}
```

第一维的长度可以为空，
其他维的长度必须指定。

实参数组与形参数组必须同构。

案例分析：销售业绩分析程序

问题

- 已知某经销商上半年3类商品每个月的价格和销量，定义函数计算销售表中各商品的销售总额，然后对该函数进行测试。

代码

```
#include <stdio.h>
#include <conio.h>

/*
    声明销售业绩分析函数
    参数n是传入的商品种类数量
    参数price是传入的各类商品的各月销售价格
    参数volume是传入的各类商品的各月销量
    参数amount是传出的各类商品的上半年销售总额
*/
void SalesAnalysis(int n, double price[][6],
                  int volume[][6], double amount[]);
```

案例分析：销售业绩分析程序

代码 (续)

```
int main(void)
{
    /* 定义二维数组保存3种商品每个月的价格 */
    double salesPrice[3][6] = {
        {23.9, 23.9, 23.9, 22.9, 21.9, 21.9},
        {89.9, 89.9, 79.9, 79.9, 69.9, 69.9},
        {65.9, 65.9, 65.9, 65.9, 65.9, 65.9}};
    /* 定义二维数组保存3种商品每个月的销量 */
    int salesVolume[3][6] = {
        {300, 290, 456, 358, 372, 366},
        {155, 134, 98, 79, 82, 94},
        {233, 215, 242, 201, 239, 251}};
    /* 定义一维数组保存3种商品半年的销售总额 */
    double salesAmount[3] = {0.0};
    int i;

    /* 调用销售业绩分析函数，计算各类商品上半年的销售总额 */
    SalesAnalysis(3, salesPrice, salesVolume, salesAmount);
}
```

案例分析：销售业绩分析程序

代码

续

```
/* 显示各类商品上半年的销售总额 */  
printf("种类 上半年销售总额（万元）\n");  
for (i=0; i<3; i++)  
{  
    printf("%-5d%-.4f\n", i+1, salesAmount[i]/10000);  
}  
  
return 0;  
}
```

案例分析：销售业绩分析程序

代码

续

```
void SalesAnalysis(int n, double price[][6],
                  int volume[][6], double amount[])
{
    int t, m;

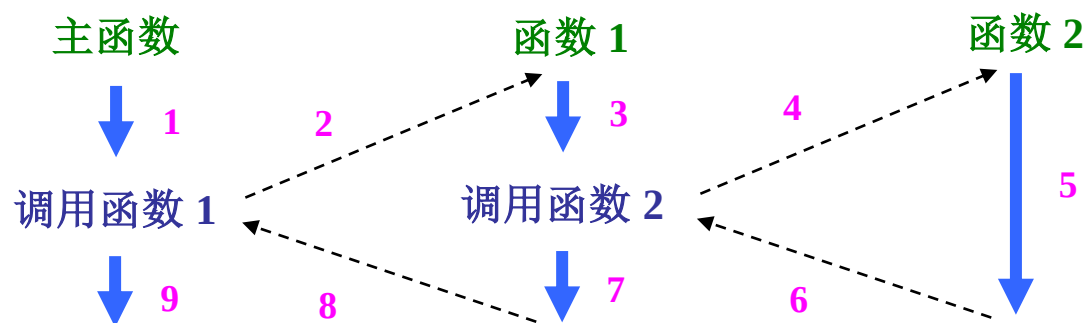
    for (t=0; t<n; t++)
    {
        double a = 0.0;
        for (m=0; m<6; m++)
            a += price[t][m]*volume[t][m];

        amount[t] = a;
    }
}
```

函数的嵌套调用

嵌套调用

✿ 在调用一个函数的过程中又调用另一个函数

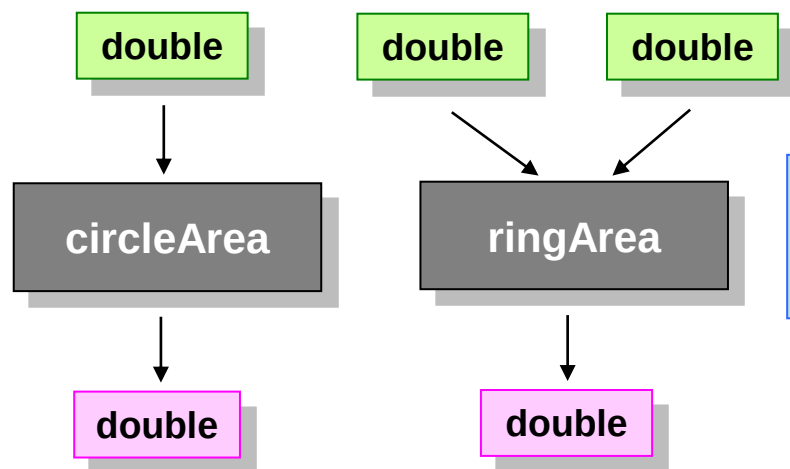
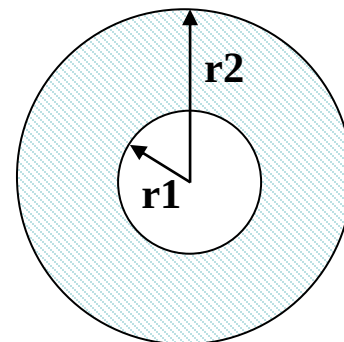


案例分析：函数的嵌套调用

计算圆环的面积

分析

- 圆环的面积 = 外圆的面积 - 内圆的面积
- 可以定义两个函数
 - **circleArea** 计算圆的面积
 - **ringArea** 计算圆环的面积



```
double circleArea(double r);  
double ringArea(double r1, double r2);
```

案例分析：函数的嵌套调用

计算圆环的面积

源代码 (cw0804.c)

```
#include <stdio.h>

#define PI 3.14

double circleArea(double r);
double ringArea(double r1, double r2);

void main() {
    double r1, r2, s;

    printf("\tInput r1, r2: ");
    scanf("%lf%lf", &r1, &r2);
    s = ringArea(r1, r2);
    printf("\n\tThe area is: %.2lf\n", s);
}
```

案例分析：函数的嵌套调用

计算圆环的面积

源代码（续）

```
double circleArea(double r) {  
    return PI*r*r;  
}  
  
double ringArea(double r1, double r2) {  
    if (r1<=r2)  
        return circleArea(r2)-circleArea(r1);  
    else  
        return circleArea(r1)-circleArea(r2);  
}
```

```
Input r1, r2: 1 2  
The area is: 9.42
```

案例分析：掷骰子

■ 掷骰子

✿ 问题

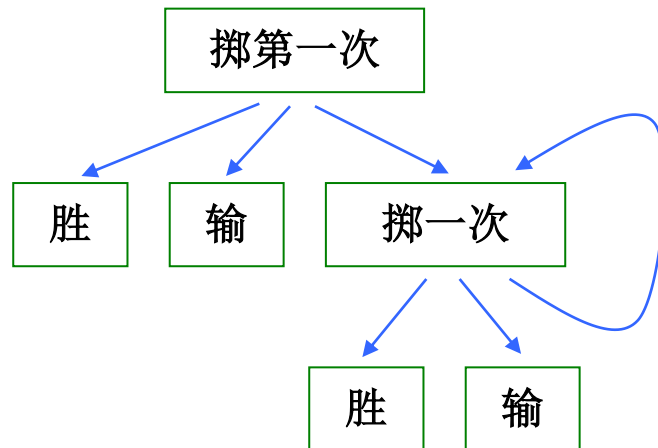
- 每个玩家掷两个骰子。每个骰子都有**6**个面。这些面中包含了**1**点、**2**点、**3**点、**4**点、**5**点和**6**点。当骰子静止下来之后，计算两个朝上的面的点数和作为本次投掷的结果。
- 如果第一次投掷的结果是**7** 或 **11**，那么这个玩家就获胜。
- 如果第一次投掷的结果是**2**、**3**或**12**，那么这个玩家就输了（即庄家获胜）。
- 如果第一次投掷的结果是**4**、**5**、**6**、**8**、**9**或**10**，那么这个结果就是该玩家的“点数”。
- 为了获胜，玩家必须继续掷骰子，直到“掷出了点数”。在掷出点数之前，如果玩家掷出了**7**，那么玩家就输了。

案例分析：掷骰子

掷骰子

初始设计

- 定义一个函数 **rollDice**，用来模拟掷一次骰子
 - 产生两个随机数（1..6），返回它们的和（点数）



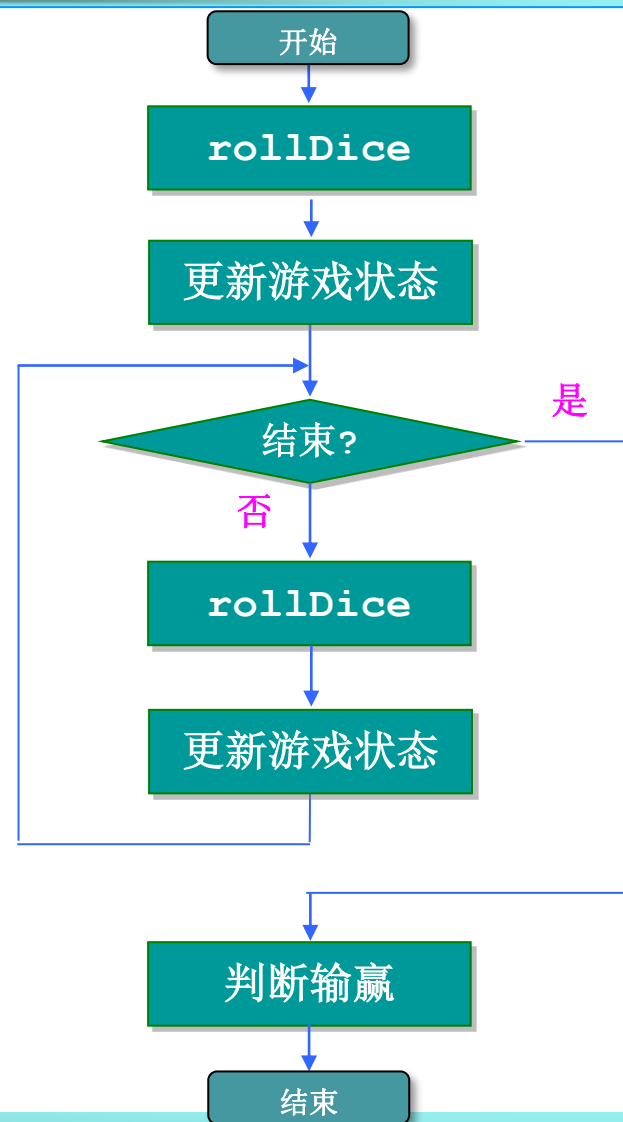
案例分析：掷骰子

掷骰子

细化设计

➤ 定义一个变量保存游戏进展的状态 **gamestatus**

- 0: 继续
- 1: 胜利 (游戏结束)
- 2: 失败 (游戏结束)



案例分析：掷骰子

掷骰子

✿ 实现 (cw0807.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int rollDice(void);

void main() {
    int gameStatus, sum, myPoint;

    srand(time(NULL));

    sum = rollDice();
```

掷第一次

案例分析：掷骰子

掷骰子

实现

```
switch(sum) {  
    case 7: case 11:  
        gameStatus = 1;  
        break;  
    case 2: case 3: case 12:  
        gameStatus = 2;  
        break;  
    default:  
        gameStatus = 0;  
        myPoint = sum;  
        printf("Point is %d\n", myPoint);  
        break;  
}
```

掷第一次之后的
结果

案例分析：掷骰子

掷骰子

实现

```
while (gameStatus == 0) {  
    sum = rollDice();  
  
    if (sum == myPoint)  
        gameStatus = 1;  
    else  
        if (sum == 7)  
            gameStatus = 2;  
}
```

继续游戏

```
if (gameStatus == 1)  
    printf("You wins!");  
else  
    printf("You loses!");  
}
```

胜负判断

案例分析：掷骰子

掷骰子

实现

```
int rollDice() {  
    int dice1, dice2, sum;  
  
    dice1 = rand()%6 + 1; //第一个骰子的点数  
    dice2 = rand()%6 + 1; //第二个骰子的点数  
    sum = dice1 + dice2;  
    printf("You rolled %d + %d = %d\n", dice1, dice2, sum);  
  
    return sum;  
}
```

- **rand()**随机产生一个0~RAND_MAX之间的整数。
- **srand()**根据给定的值初始化随机数产生器。
- **time()**获得当前的系统时间，并转换为一个长整数。

小结

- 函数可以作为大型程序的组成模块。
- 每个函数应该实现某个明确的功能。
- 使用参数可以向函数传递数据，通过return让函数返回一个数据。
- 使用函数原型声明函数，以便编译器检查函数调用时所传递的参数个数及类型是否正确。